Computer Organization:

Unit I: Basic Structure of Computers

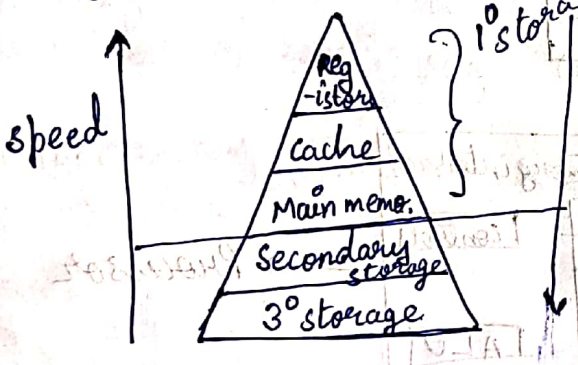
- **Functional Units:**

I/O devices — Memory — { ALU [Arithmetic and Logic Unit] / Control unit } Processor

* **Information handled by a computer:**
  - → Instructions/machine instructions: Govern transfer of info and specify arithmetic and logic
  - → Data: Source program, used as operands
  - → Encoded in binary - 0 and 1

* **Memory unit:**
  - → Stores programs & data
  - → Two classes → Primary storage / Secondary storage.

Memory hierarchy:



cache - SRAM

Main memory - DRAM

* Operating system transfers data from secondary storage and Main memory.
* Main memory is in form of matrix
* Each row is called location & has unique adress.
  Load-Read, store - write → RISC
* Bus = Group of wire
* Has Large number of semiconductor storage cells.

RAM + ROM → Has BIOS program.
↳ Is erased after used, i.e, volatile

Scanned with CamScanner

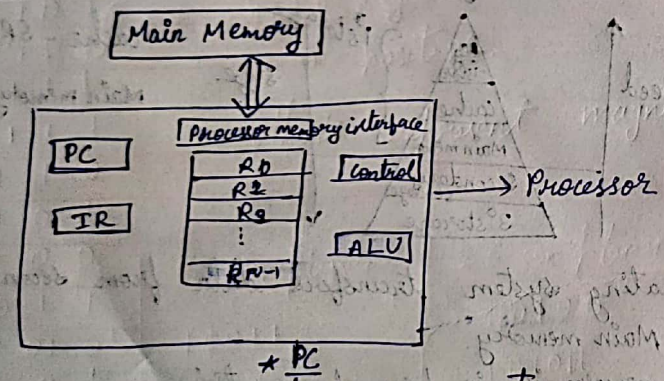
* ALU :
Most operations takes place in ALU.

* Control unit :
→ Controls all computer operations
→ Transmits control & timing signals.

• Basic Operational functions :

* Sample instruc^n :
→ Load R2, LOC : R2 ← (LOC) → Register transfer Notation
Load → Read, LOC → Location variable
this means copy data from LOC to register2
→ Add R4, R2, R3 : R4 ← [R2] + [R3]
→ Store R4, LOC : [LOC] ← R4
Store → write

* Connection b/w processor & memory :



IR :
↓
Instruction register
↳ stores instruc^n.

* PC
↳ Program counter
↓
stores address of instruc^n.

processor memory interface.
↓ ↓
MAR [Memory Address register]
MDR [Memory Data Register]

* Buses are three types
→ Address bus → unidirectional bus
→ Control bus } Bi directional bus
→ Data bus }

* PC is incremented by word length of processor.
↳ points to 1st instruc^n

Step I : PC sends address to MAR in interface and control signal read
Step II : First instruc^n stored in Processor mem. Interface, stores in IR.
Step III : Sent to Decoder, decodes instruc^n sent to ALU
Step IV : Operation executed in ALU.
Step V : Store the result back. → general purpose registers → memory [write signal]
Step VI : PC is set to next instruc^n during execution.

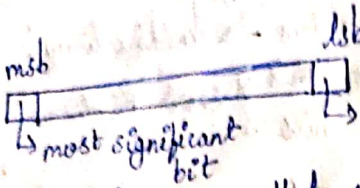
* Decoder
→ Recognises the instruc^n.
→ Fetches operand.

* when processor receives INTR [Interrupt request], it stops execution of current program, executes ISR [Interrupt service routine], completes and comes back to main program.

Scanned with CamScanner

Number Representation:

- Sign magnitude
- One's complement
- Two's complement



They are called so, because of the higher power of 2, at that bit position.

Total value = 16 [2^4]

4 bits → range = 0 to 15 unsigned.

= −8 to +7

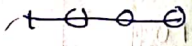
Sign magnitude:

0	0 0 0 0
1	0 0 0 1
2	0 0 1 0
3	0 0 1 1
4	0 1 0 0
5	0 1 0 1
6	0 1 1 0
+7	0 1 1 1
−7	1 1 1 1
−6	1 1 1 0
−5	1 1 0 1
−4	1 1 0 0
−3	1 0 1 1
−2	1 0 1 0
−1	1 0 0 1
−0	1 0 0 0

Most significant bit represents sign

- 1 - negative
- 0 - positive.

Other's represent magnitude



One's complement:

All the bits are toggled, except most significant bit.

1's complement = $(2^n - 1)$ − num

n → no. of bits.

num → Magnitude.

Ex: −7 1's complement =

7 = 0111 = 1000

1's complement of (7) = $(2^4 - 1) - (+7)$

= 15 − 7 = 8 = 1000

1's complement of (5) = $(2^4 - 1) - 5$

= 15 − 5 = 10

Because of diff. representation of 0 and −0, 1's complem & sign magnitude are not used.

Two's complement:

Move from lsb to msb, till 1 is found, toggle the bits before the 1.

Scanned with CamScanner

		1's comp.	2's comp
0	0000	1000	1001
1	0001	1001	1010
2	0010	1010	1011
3	0011	1011	1100
4	0100	1100	1101
5	0101	1101	1110
6	0110	1110	1111
7	0111	1111	0000
-7	1111		
-6	1110		
-5	1101		
-4	1100		
-3	1011		
-2	1010		
-1	1001		
-0	1000		

Range: -2^{n-1} to $2^{n-1}-1$

$(2^n - num) = 2^s$ complement

1.) Represent using 6 bit representation:

(i) -12
(ii) -21
(iii) 17
(iv) -30

	Sign magnitude	1's complement	2's complement
(i)	101100	110011	110100
(ii)	110101	101010	101011
(iii)	010001	010001	010001
(iv)	111110	100001	100010

* n bit word.
Range of values for unsigned = 0 to $2^n - 1$
Range of values for signed = -2^{n-1} to $2^{n-1} - 1$.

Addition & Subtraction using 2's complement:

1) 4 + 3 = 0100 + 0011 = 0111 = 7//

2) -4 + (-3) = -0100 + (-0011)

2's complement of -4 = 1100
-3 = 1101

1001 = -0110 + 1
carry = ignore = -0111
= -7//

* 2's complement most common choice for integer number systems

3) 4 + (-3)

Taking 2's complement of 3, 0011
= 1100 + 1
= 1101

0100 + 1101 = 10001
ignore → 1//

Overflow condition occurs when two numbers of same sign are added.

Can be found, when the result has opp. sign of the two numbers.

Scanned with CamScanner

(8) Convert the following to 2's complement binary numbers in 5-bits & add them. State whether overflow occurs in each case.

(i) 7 and 13

$2|7$ $2|3-1$ $2|1-1$

$2|13$ $2|6-1$ $2|3-0$ $1-1$

```
 00111
+01101
 01010   = 10
```

No overflow

(ii)

```
 00111
 01100 1
 01010 0
```

Overflow

(iii) -3 and -8

```
 11101
+11001
 10110
```

$(-01010)_2 = -11$

No overflow

```
 00011
 10011
 11100
```

$2|8$ $2|4-0$ $2|2-0$ $1-0$

```
 0111
-1000
```

```
 0010
 1011 +
 1000
```

(iii) -12 and -7

$2|12$ $2|6-0$ $2|3-0$ $1-1$

$2|7$ $2|3-1$ $2|1-1$

```
 01100
 10100
+01001
 11101
-00010+1
```

No overflow

$= -00011 = -3$

```
 01100   12
 10100  -12
```

(iv) Subtract -6 from 4 using 4 bit 2's complement.

$4-(-6) = 4+6$

```
 0100
 0110
 1010
```

Overflow

(i) -5 from -7 = 4 bit

$2|5$ $2|2-1$ $1-0$

$2|7$ $2|3-1$ $1-1$

1001

```
 1011
+1001
 10100
```

carry

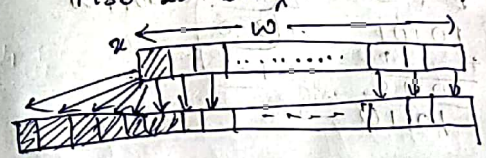
Overflow

$-7-(-5)$

$-7-2^3$ — 2^3 4's

Sign extension:

MSb is extended all the other higher order bits



Scanned with CamScanner

(10)

5-bit, 2's complement

Perform addition & subtraction in each pair

(i) 6 and 14

(ii) -10 and -13.

Ans: (i) 6 + 14

```
  11
  00110
+ 01110
  10100
```

Overflow.

2's complement of -14 = 10010

6 - 14

```
  11
  00110
+ 10010
  11000
```

2's complement of 11000, 0111 1001 + 00100

= -01000 = -8 //

No overflow.

(ii) -10 and -13

2's complement of -10 is 10110

2's complement of -13 is 10011

```
  10110
+ 10011
  01001   (1 = carry)
```

Overflow occurs

Using 6-bits, 2's complement of -10 is 110110

—— 11 —— -13 is 110011

```
  110110
+ 110011
  101001   (1 = carry)
```

= -010111 (weights 32 16 8 4 2 1)

= -23

No overflow

(11)

-10 - (-13)

2's complement of -10 is 10110

—— 11 —— -13 is 10011

-10 - (-13) = 10110 - (10011)

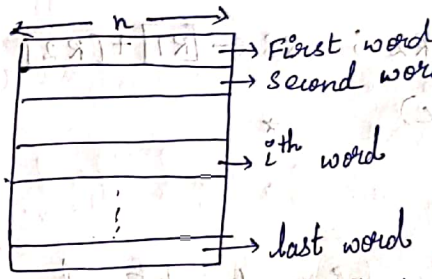
2's complement is 01101

∴

```
  10110
+ 01101
  00011   (1 = carry)
```

= 3 //

- Memory Locations, Addresses & Operation:



1 byte = 8 bits.

Memory: n = word length such as

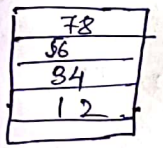
n=8 for 8-bit processor.

Memory is byte addressible.

Data stored in memory: (Intel processors)

- Little Endian - Least significant bit is stored in lower order memory
- Big Endian - Higher order memory stores lsb.

Little Endian:



Big Endian:



Scanned with CamScanner

(12) In RISC:

(Read) Load R1, LOC A

(write) Store R1, LOC A

Add R1, R2, R3

C = A+B in RISC machine – Both operands should be in registers

Step I: (Read A)

Load R1, A

Step II: (Read B)

Load R2, B

Step III: Add R3, R1, R2 ; R3 ← [R1] + [R2]

Step IV: (write R3 into C)

Store R3, C

1.) Consider high level assignment statement

X = A * B + C / D , where

X, A, B, C and D are memory locations.

write a program that computes X in RISC style.

Load R1, A
Load R2, B
Multiply R3, R1, R2 ; R3 ← [R1] * [R2]
Load R4, C
Load R5, D
Divide R6, R4, R5 ; R6 ← [R4] / [R5]
Add R7, R3, R6 ; R7 ← [R3] + [R6]
Store R7, X

Instruction, Execution and straight line sequencing:

On the basis of no. of operands, the instruction are classified as:

0- address
1 - address
2 - address
3 - address

C = A + B

Load R2, A
Load R3, B
Add R4, R2, R3 → 3 - address instruction
Store R4, C

2 - address:

Add R2, R3 ; R2 ← [R2] + [R3]

1 - address:

Add R3 ; A ← [A] + [R3]

A - Accumulator [default value register]

0 - address;

Push / Pop : → the element in top of stack is removed
→ Accumulator is stored above the stack.

Arithmetic operation is performed on the top of two element of stacks.

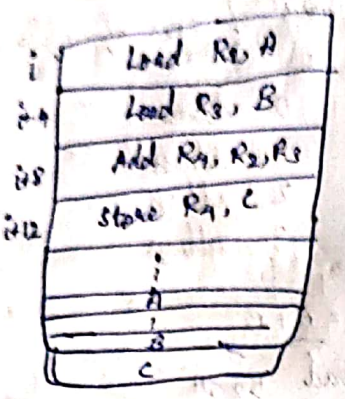
Scanned with CamScanner

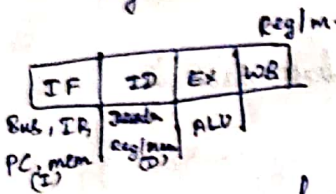
Assumptions

* 32-bit processor
* Byte addressable memory

Program executed is two phases:

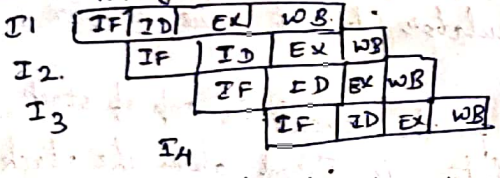
- Instruction fetch
- Program execution.





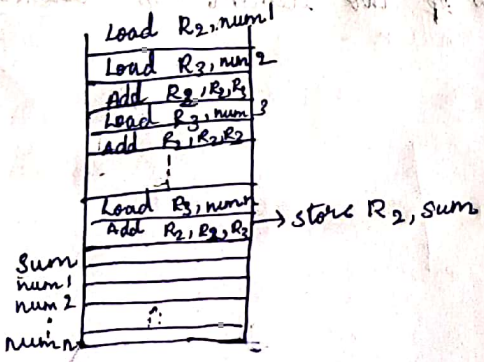
Memory → D-cache, I-cache

Pipelines: Overlapping basic operational steps of instruction execution

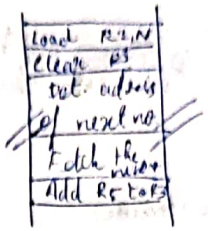


This leads to lowering of clk cycles used for execution of the programs, increasing performance.

Add list of 'n' nos.



Branching:



$R_2 \leftarrow N$

Branch is taken by checking contents of registors in RISC machines

Load R_2, N	$R_2 \leftarrow N$	
Clear R_3	$R_3 \leftarrow 0$	
Determine address of next no.	$R_4 \leftarrow$ address of	1st no.
Load the next no. to R_5	$R_5 \leftarrow$ [num1]	
Add R_5 to R_3.	$R_3 \leftarrow [R_3] + [R_5]$	
Sub $R_2, R_2, \#1$	$R_2 \leftarrow R_2 - 1$	
Branch if $[R_2] > 0$	LOOP	

Condition codes:

CISC machines have a flag registor:

Scanned with CamScanner

Generating memory address:

EA = Effective address (address of operand)

Addressing modes

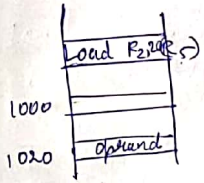
Name	Syntax	
Immediate	#value	operand = value
Registers	R_n	R_n ← operand, EA = R_n
Absolute	LOCA	EA = LOCA
Register Indirect [Same as pointers]	(R_n)	$EA = [R_n]$
Indexing	$X(R_i)$	$EA = X + [R_i]$
Base with Index	$(R_i)(R_j)$	$EA = [R_i] + [R_j]$

A = *B

Load R_2, B ; EA = B
Load R_3, (R_2) ; EA = $[R_2]$
Store R3, A ; EA = A

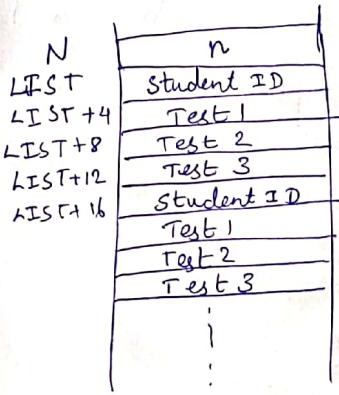
Loop using indirect addressing mode

	Load	R_2, N	size of list
	clear	R_3	Sum initialized to 0.
	Move	R_4, #NUM1	address of first number
LOOP:	Load	R_5, (R_4)	Get next number
	Add	R_3, R_3, R_5	Add number to sum.
	Add	R_4, R_4, #4	Incrementing pointer
	subtract	R_2, R_2, #1	Decrement counter
	Branch_if_[R_2]>0	LOOP	Branch back if not finished.
	Store	R_3, SUM	Store final sum.



1000 R5

After execution, R5 remains 1000, as it only applies to E.A.



Use the following registers:
R_2 ← pointer
R_3 ← Partial sum of Test 1
R_4 ← —u— of Test 2
R_5 ← Test 3
R_6 ← value of n

Scanned with CamScanner

```
        Move    R2, #LIST
        Clear   R3
        Clear   R4
        Clear   R5
        Load    R6, N
LOOP:   Load    R7, 4(R2)
        Add     R3, R3, R7
        Load    R7, 8(R2)
        Add     R4, R4, R7
        Load    R7, 12(R3)
        Add     R5, R5, R7
        Add     R2, R2, #16
        Subtract R6, R6, #1
        Branch-if-[R6]>0 LOOP
        Store   R3, SUM1
        Store   R4, SUM2
        Store   R5, SUM3
```


Assembly Language:

* Set of symbolic names & rules for their use.
* Assembler program – source program to machine level language
* Assembler directives are used to specify other information needed to translate source pgm into object pgm.

Assembler
- → How to interpret names
- → Place instructions in memory
- → Place data operands in memory.

EQU → Assign value to name.
ORIGIN → Tells the memory location where data is to be placed.
DATAWORD → loads value to variables.
RESERVE → Reserves memory block
END → Record End of source pgm.
START → Begin execution of pgm.

Adding list of n numbers:

```
        ORIGIN  100
        LD      R2, N
        CLR     R3
        MOV     R4, #num1
LOOP:   LD      R5 (R4)
        ADD     R3, R3, R5
        ADD     R4, R4, #4
        SUB     R2, R2, #1
        BGT     R2, R0, LOOP
        ST      R3, Sum
        ORIGIN  200
SUM     RESERVE 4
N       DATAWORD 150
NUM1    RESERVE 600
        END
```

Scanned with CamScanner

Two pass Assembler:

- 1st pass: Symbol table is built
- 2nd pass: Replaces the name with value.

Loader:

- Loads object prg. to main memory.
- Starts executing program.

Debugger:

- Allows to see status of program or register.

CISC instructions:

* No load-store instructions to access memory.
* All the data transfer (move) & Arithmetic logic instructions can access ~~xxx~~ memory
* Variable length instructions.

Operation destination, source

For C = A+B :

Move R_2, A ; $[R_2] \leftarrow A$
Move R_3, B ; $[R_3] \leftarrow B$
Add R_3, R_3, R_2 ; $[R_3] \leftarrow [R_3]+[R_2]$
Move C, R_3 ; $C \leftarrow [R_3]$

(or)

Move R_2, A ; $[R_2] \leftarrow A$
Add R_2, B ; $[R_2] \leftarrow [R_2]+[B]$
Move C, R_3 ; $C \leftarrow R_3$

Move = for immediate value.
Load = for memory location.

Additional addressing modes:

* Auto increment	(Ri)+	EA = [Ri], access operands, Ri = [Ri] + no. of word length
* Auto decrement	-(Ri)	EA = [Ri] - 4, operand is accessed
* Relative	X(PC)	EA = X + [PC]

↳ Automatically executed in branching instruc^n.
X – backward jump – -ve.
X' – forward jump – +ve.

Registers R_4 & R_5 contain decimal numbers 2000 & 3000
Each of the following addressing modes is used to access memory. what is EA in each case.

ⓐ 12(R_4) : Indexing ; EA = 12 + [R_4] = 2012

ⓑ (R_4, R_5) : Base with index : EA = 2000 + 3000 = 5000

ⓒ 28(R_4, R_5) : Indexing : EA = 28 + 2000 + 3000 = 5028

ⓓ (R_4)+ : Auto increment : EA = 2000, R_4 = 2004

ⓔ -(R_4) : Auto decrement : EA = 2000 - 4 = 1996

write a CISC style program that computes the exp.

SUM = 580 + 68400 + 180000
where sum is memory location.

Move R_2, 580
Move R_3, 68400
Load R_2, #580
Load R_3, #68400

Move R_2 #580
Move R_3 #68400
Add R_3, R_2
Add R_3, #180000
Move SUM, R_3
//Store R_3, SUM

Scanned with CamScanner

CISC.

2) ANS = A × B + C × D

```
Move   R2, A
Move   R
Multiply R2, B
Move   R3, C
Multiply R3, D
Add    R2, R3
Move   ANS, R2.
```

3) Write a CISC style program to add list of n numbers.
Consider address of 1st array element to be Num1.

```
      Move   R2, n
      Clear  R3
      Move   R4, #Num1
LOOP: Move   R5, (R4)+
      Add    R3, R5
      Subtract R2, #1
      Branch_if_[R2]>0 LOOP
      Move   Sum, R3
```

Branch is used
Register contents
not checked
only Flags
are checked.

Scanned with CamScanner